J. Symbolic Computation (1993) **15**, 27-48

Implementing CLP(𝔹) using the Connection Theorem Proving Method and a Clause Management System

GREG SIDEBOTTOM

Department of Computing Science, Simon Fraser University
Burnaby, British Columbia V5A 1S6, Canada

Constraint Logic Programming (CLP) is an extension of logic programming to include efficient constraint processing algorithms for specialized computational domains. CLP(𝔹) is a CLP language which processes constraints in Boolean algebra. The CLP(𝔹) language can be used to solve many set theoretic and propositional logic problems including digital circuit design, analysis, and verification. This paper formally describes a new way to implement CLP(𝔹) using the connection theorem proving method (CM) and a clause management system (CMS). The CM and a CMS provide a simple and efficient way to implement CLP(𝔹).

1 Introduction

Constraint Logic Programming (CLP) is an extension of logic programming to include efficient constraint processing algorithms for specialized computational domains. These specialized domains are added in a way that preserves the attractive semantic properties that logic programming already possesses (Jaffar & Lassez, 1987). CLP(𝔹) is a CLP language which contains constraints in Boolean algebra. The CLP(𝔹) language can be used to solve many set theoretic and propositional logic problems including digital circuit design, analysis, and verification (Buttner & Simonis, 1987; Sakai & Aiba, 1989; Dincbas *et. al.*, 1988; Colmerauer, 1990). This paper formally describes *Boole*, an experimental implementation of CLP(𝔹) using the connection theorem proving method (CM) (Bibel, 1983; Bibel, 1987) to test the satisfiability of constraints and a clause management system (CMS) (Kean & Tsiknis, 1990; Reiter & deKleer, 1987) to simplify

0747-7171/93/010027+22 $08.00/0 © 1993 Academic Press Limited

answer constraints. This paper attempts to show that the CM and a CMS provide a simple and efficient way to implement CLP(**B**).

The next section gives an overview of CLP and relates some other CLP(**B**) systems to Boole. Section 3 gives the background necessary to formally describe how Boole is implemented. Section 4 describes how the CM and a CMS are used in Boole. Finally, some conclusions are given and future work is described in section 5.

2 Constraint Logic Programming

The CLP scheme defined by Jaffar & Lassez (1987) formally specifies some semantic properties true of all instances of the scheme. The CLP(**D**) language augments a definite clause logic programming language with a *constraint system* **D**. A constraint system contains a domain and a language for describing that domain. Constraints are composed from function and predicate symbols which are interpreted in the constraint system's domain. For example, CLP(**R**) (Jaffar & Michaylov, 1987) is an instance of the CLP scheme which includes numeric equalities and inequalities on terms involving numeric function symbols. Atomic formulæ in the bodies of clauses may be either uninterpreted atoms or constraints. CLP programs are executed using SLD-resolution (Lloyd, 1984) except unification is generalized to constraint solving and answer substitutions are generalized to answer constraints. If the constraint system satisfies certain reasonable properties, the correspondence between the operational, logical, and least-fixed-point semantics in logic programming (Lloyd, 1984) carries over to the new CLP language.

Many CLP systems, including Prolog, Prolog II (Colmerauer, 1984), CLP(**R**) (Jaffar & Michaylov, 1987), and CLP(Σ^*) (Walinsky, 1989) are instances of this scheme. Some systems, such as CHIP (Van Hentenryck, 1989) and Prolog III (Colmerauer, 1990), are based on different theoretical models, but most of their constraint processing domains are in the CLP scheme. Saraswat (1990) formally studies several constraint processing domains in the CLP framework.

To be an instance of the CLP scheme, the constraint system must be *satisfaction-complete* and *solution-compact*. A constraint system is satisfaction-complete if the satisfiability of every constraint is decidable. A constraint system is solution-compact if every element of the constraint domain can be specified by a constraint and the negation of every constraint

can be specified. It is easy to verify that CLP(𝔹) is an instance of the scheme. 𝔹 is satisfaction-complete because there are many algorithms for testing the satisfiability of Boolean constraints. 𝔹 is solution compact since the system contains equalities and constants denoting both elements of 𝔹 and because 𝔹 contains a symbol denoting negation[1].

After identifying a useful constraint system which satisfies the requirements imposed by the CLP scheme, the problem is to implement efficient algorithms for processing the constraints in the system. Three widely documented logic programming languages which provide processing for Boolean constraints are: Prolog III (Colmerauer, 1990), CHIP (Dincbas *et. al.*, 1988), and CAL (Sakai & Aiba, 1989). They use two basic methods for processing Boolean constraints. Prolog III bases its Boolean constraint processing algorithms on resolution theorem proving (Boi & Benhamou, 1988) while CHIP and CAL solve Boolean constraints algebraically using properties of the Boolean ring (Buttner & Simonis, 1987; Sakai & Aiba, 1989).

Like Prolog III, Boole uses theorem proving techniques. However, the CM has an important advantage over resolution theorem proving. Resolution theorem proving requires that Boolean constraints initially be converted to conjunctive normal form[2] (CNF). This results in a constraint which can be (and often is in practice) exponentially larger in size than the original constraint. The CM operates on constraints in negation normal form (NNF). Constraints involving 'and', 'or', and 'not' can be converted to NNF without changing the number of 'and' and 'or' symbols.

Boole, like Prolog III, computes answer constraints in a minimal normal form expressed in terms of the 'and', 'or', and 'not' function symbols. Boole uses a minimal disjunctive normal form (DNF) and Prolog III uses a minimal CNF. Boole uses the CM extended with a CMS (Reiter & deKleer, 1987; Kean & Tsiknis, 1990) and the Quine-McCluskey (McCluskey, 1986) minimization method to simplify Boolean answer constraints to a minimal DNF.

---

[1]𝔹 is used here, somewhat ambiguously, to refer to both the whole constraint system and the constraint domain which is the intended model for formulæ in that system.

[2]Formal definitions of disjunctive, conjunctive, and negation normal form are given in section 3.

3 Background

This section gives the background necessary to describe how Boole implements CLP(**B**). Section 3.1 describes the Boolean Constraint system, section 3.2 introduces the CM, section 3.3 defines prime implicants, and section 3.4 describes the CLP(**B**) language and its operational semantics.

3.1 THE BOOLEAN CONSTRAINT SYSTEM

We describe the constraint system of CLP(**B**) using sorted logic (Saraswat, 1990) with the single sort:

$$\mathsf{Bool} = \{0, 1\},$$

the set of two Boolean *constant* symbols. The vocabulary for this constraint system contains the set Var of *variable* symbols ranging over Bool[3], the *function* symbols:

$$' : \mathsf{Bool} \rightarrow \mathsf{Bool},$$

$$+, \cdot : \mathsf{Bool} \times \mathsf{Bool} \rightarrow \mathsf{Bool},$$

and the *relation* symbol:

$$= : \mathsf{Bool} \times \mathsf{Bool}^{4}.$$

The model for these symbols is the standard model **B** of Boolean algebra. A formal definition of **B** can be found in (McCluskey, 1986). Let ι be the function which maps elements of the language into their interpretations in **B**. Then the symbols have the following interpretations:

$$\iota(0) = \textbf{false},$$

[3]We use the Edinburgh Prolog convention of denoting variable symbols by strings of alphanumeric characters starting with an uppercase letter. Later, uninterpreted relation symbols are denoted by strings of alphanumeric characters starting with an lowercase letter.

[4]It will always be clear from the context whether the symbol '=' denotes the relation symbol or mathematical equality.

$$\iota(1) = \textbf{true},$$

$$\iota(') = \textbf{not},$$

$$\iota(+) = \textbf{or},$$

$$\iota(\cdot) = \textbf{and}, \text{ and}$$

$$\iota(=) = \textbf{equal}.$$

The set Term of terms and the set Con (for *constraints*) of atomic formulæ over Term are defined in the usual way. We frequently use a term t to stand for the constraint $t = 1$.

Where convenient, the operators:

$$\Sigma, \Pi : 2^{\mathsf{Con}} \rightarrow \mathsf{Con}.$$

are used to denote constraints equivalent to the disjunction and conjunction of a given set of constraints, respectively. Formally, we define them recursively by:

$$\Sigma\emptyset = 0,$$

$$\Sigma(\{c\} \cup C) = c + \Sigma C,$$

$$\Pi\emptyset = 1, \text{ and}$$

$$\Pi(\{c\} \cup C) = c \cdot \Pi C.$$

Variable symbols and their negations are called *literals*. Formally, the set Lit of literals is defined by

$$\mathsf{Lit} = \{\mathsf{V}, \mathsf{V}' \mid \mathsf{V} \in \mathsf{Var}\}.$$

The pair $\{\mathsf{V}, \mathsf{V}'\}$ is called a *complementary* pair of literals and each is the other's *complement*. Literals of the forms V and V' are called *positive* and *negative* literals, respectively. Given a finite set L of literals, ΣL and ΠL are called *clauses*. ΣL is a *disjunctive* clause and ΠL is a *conjunctive* clause. Let $L_1,\ldots,L_p$ be finite sets of literals. Then a constraint is in CNF if it is of the form $\Pi\{\Sigma L_1,\ldots,\Sigma L_p\}$ and DNF if it is of the form $\Sigma\{\Pi L_1,\ldots,\Pi L_p\}$.

The *dual* of $c \in$ Con, which is denoted by *dual*(c), is the result of replacing every '+' symbol by '·' and every '·' by '+'. Later, we apply *dual* to sets of constraints, meaning the result of replacing every constraint in the set by its dual. The notation *neglit*(c) is used to represent the result of replacing every literal in c by its complement. Note that for a literal l, *neglit*(l) is the complement of l and $\{l, neglit(l)\}$ is a complementary pair of literals.

For convenience,

$$\Rightarrow, \Leftrightarrow : \mathsf{Bool} \times \mathsf{Bool} \rightarrow \mathsf{Bool}$$

are used to abbreviate terms denoting implication and equivalence, respectively. That is, for $t_1, t_2 \in$ Term,

$$t1 \Rightarrow t2 = t1' + t2 \text{ and} \tag{1}$$

$$t_1 \Leftrightarrow t_2 = (t_1' + t_2)\cdot(t_1 + t_2'). \tag{2}$$

To reduce the number of parentheses, all infix binary symbols are assumed to associate left to right and the precedence of the symbols decreases according to the order:

$$', \cdot, +, \Rightarrow, \Leftrightarrow, =.$$

Let c be a constraint, $\mathsf{X}_1,\ldots,\mathsf{X}_p$ be distinct variables, $b_1,\ldots,b_p \in$ Bool, and $\sigma = [\mathsf{X}_1\backslash b_1,\ldots,\mathsf{X}_p\backslash b_p]$. Then σ is called a *substitution* and $c\sigma$ denotes the constraint resulting from replacing every occurrence of X_k in c by b_k ($1 \le k \le p$). The notation *vars*(c) is used to denote the set of variable symbols in a constraint c. We say that σ is a *grounding substitution for c* if $vars(c) \subseteq \{\mathsf{X}_1,\ldots,\mathsf{X}_p\}$. Where we call σ a grounding substitution and apply it to a constraint c, we implicitly mean that σ is a grounding substitution *for c*.

Since the domain of each variable is one of two possible values, we can define existential and universal quantification in terms of disjunctions and conjunctions of constraints. The function symbols:

$$\exists, \forall : \mathsf{Var} \times \mathsf{Con} \rightarrow \mathsf{Con}$$

are used to denote disjunctions and conjunctions as follows:

$$\exists \mathsf{X}\ c = \Sigma\{c[\mathsf{X}\backslash 0], c[\mathsf{X}\backslash 1]\} \text{ and}$$

$$\forall X\ c = \Pi\{c[X\backslash 0],\ c[X\backslash 1]\}.$$

Given a constraint c and distinct variables $X_1,\ldots,X_p$, the function symbols:

$$\exists, \forall : 2^{\mathsf{Var}} \times \mathsf{Con} \rightarrow \mathsf{Con}$$

are used to denote the result of applying the operations defined by:

$$\exists\{X_1,\ldots,X_p\}\ c = (\exists X_1 \ldots (\exists X_p\ c)\ldots) \text{ and}$$

$$\forall\{X_1,\ldots,X_p\}\ c = (\forall X_1 \ldots (\forall X_p\ c)\ldots).$$

We use the convention that $\exists\emptyset\ c = \forall\emptyset\ c = c$. The function symbols:

$$\exists, \forall : \mathsf{Con} \rightarrow \mathsf{Con}.$$

are used to denote the result of applying the operations defined by:

$$\exists c = \exists vars(c)\ c \text{ and}$$

$$\forall c = \forall vars(c)\ c.$$

They are known as the existential and universal closures of c, respectively.

A grounding substitution σ *satisfies* a constraint c if $\mathbb{B} \models c\sigma$. A constraint c is *satisfiable* if there is at least one grounding substitution which satisfies it and *valid* if every grounding substitution satisfies it. Note that this is the same as saying a constraint c is satisfiable if $\mathbb{B} \models \exists c$ and valid if $\mathbb{B} \models \forall c$. Note also that c is satisfiable if and only if c' is not valid. Two constraints, c_1 and c_2 are *equivalent* if for all grounding substitutions σ, σ satisfies c_1 if and only if σ satisfies c_2. Note that this is the same as saying c_1 and c_2 are equivalent if $c_1 = c_2$ is valid.

3.2 The Connection Theorem Proving Method

The CM as described here applies to a subset of the constraints called *negation normal form* (NNF) constraints or *matrices*[5]. The CM proves that a matrix is valid by showing

[5]It should be pointed out that [Bibe87] defines matrices to be a proper subset of constraints in NNF.

no *path* through it is *uncomplementary*. This section defines the terms matrix, path, and uncomplementary.

A constraint is a matrix if it contains only the function symbols "', '+', and '·', and "' is only applied to variables. More formally, the set Mat of matrices is defined by the closure of the following rules[6]:

M1. if $l \in$ Lit then $l \in$ Mat, and

M2. if M is a finite subset of Mat then $\Sigma M \in$ Mat and $\Pi M \in$ Mat.

Notice that both 0 and 1 are matrices because when $M = \emptyset$ in rule M2, $\Sigma\emptyset = 0$ and $\Pi\emptyset = 1$. Any constraint can be translated to an equivalent matrix by using (1) and (2) to remove all instances of '⇒' and '⇔', driving all negations into the variables using de Morgan's theorems:

$$(\Sigma\{t_1,\ldots,t_p\})' = \Pi\{t_1',\ldots,t_p'\}, \text{ and} \tag{3}$$

$$(\Pi\{t_1,\ldots,t_p\})' = \Sigma\{t_1',\ldots,t_p'\}, \tag{4}$$

and removing all instances of double negation using the theorem:

$$t'' = t. \tag{5}$$

For instance, the constraint

$$(A \cdot B)' \Leftrightarrow (A'+B'), \tag{6}$$

which is equivalent to an instance of (4), is equivalent to the following matrix:

$$((A \cdot B) + (A'+B')) \cdot ((A'+B') + (A \cdot B)). \tag{7}$$

A path through a matrix m is a subset of the literals in m. The set $paths(m)$ of paths through m is defined by the closure of the following rules:

P1. $paths(l) = \{\{l\}\}$ if $l \in$ Lit,

P2. $paths(\Sigma\{m_1,\ldots,m_d\}) = \{P_1 \cup \ldots \cup P_d \mid P_1 \in paths(m_1),\ldots,P_d \in paths(m_d)\}$, and

[6]Here we are using the convention that a term t stands for the constraint $t = 1$. Technically, Mat is a set of terms and the set of matrices is $\{m = 1 \mid m \in \text{Mat}\}$.

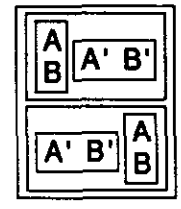
P3. $paths(\Pi\{m_1,\ldots,m_d\}) = paths(m_1) \cup \ldots \cup paths(m_d)$.

Informally, a path must go through every disjunct in each disjunction and exactly one conjunct in each conjunction encountered in the traversal of the matrix. Note that this definition implies that

$$paths(0) = paths(\Sigma\emptyset) = \{\emptyset\} \text{ and}$$

$$paths(1) = paths(\Pi\emptyset) = \emptyset.$$

To help visualize paths, it is useful to depict matrices by a set of nested boxes, one for each conjunction and disjunction, with disjunctions laid out horizontally and conjunctions laid out vertically. For instance, matrix (7) above is depicted by:



(8)

Paths are obtained by traversing the diagram horizontally, going through all components of each horizontal box and through one component of each vertical box encountered. Thus, the set of distinct paths through (8) is

$$\{\{A, A', B'\}, \{B, A', B'\}\}. \tag{9}$$

A path is *uncomplementary* (UC) if it contains no complementary pair of literals. Let $UCpaths(m)$ be the set of UC paths through a matrix m. Notice that $UCpaths(m) \subseteq paths(m)$ for all matrices m. Bibel (1987) proves that m is valid if and only if it $UCpaths(m) = \emptyset$. Thus, (8) is valid since (9) contains no UC paths. Equivalently, m is satisfiable if and only if $UCpaths(m') \neq \emptyset$. Bibel (1987) gives efficient algorithms for searching for UC paths.

It should be noted that when a constraint is converted to an equivalent matrix, the time required to search for UC paths decreases with the number of paths in the matrix. Thus, it is desirable to simplify matrices, minimizing the number of paths through them. For instance suppose we use rule (2) above to remove the '$\Leftrightarrow$' in $A \Leftrightarrow B$. The resulting matrix, $(A'+B)\cdot(A+B')$, contains four paths. However, $A \Leftrightarrow B$ is also equivalent to

(A·B)+(A'·B'), which contains only two paths. But this simplification only works for occurrences of the '⇔' connective which are nested within an even number of '' connectives (either explicitly introduced or implicitly introduced by a '⇒' connective). Occurrences of the '⇔' connective within an odd number of negations should be left alone because driving the negations into the variables puts them in the form with only two paths. Bibel (1987) gives several other matrix simplification rules. In this paper, we make no attempt to simplify matrices.

3.3 PRIME IMPLICANTS

Let K and L be finite sets of literals. Then ΠL *subsumes* ΠK if L is a subset of K. A conjunctive clause ΠL is an *implicant* of a constraint c if ΠL is satisfiable and $\Pi L \Rightarrow c$ is valid. ΠL is a *prime implicant* of c if ΠL is an implicant of c and there is no implicant of c different from ΠL which subsumes ΠL. A *complete* set of implicants for a constraint c is a set I of implicants for c such that $\Sigma I \Leftrightarrow c$ is valid. A complete set I of implicants for c is *redundant* if some proper subset J of I is a complete set of implicants for c. $PI(c)$ will be used to denote the set of prime implicants for a constraint c. McCluskey (1986) shows that $PI(c)$ is unique and is a complete set of implicants for c and gives an algorithm to select a non-redundant complete subset Min of $PI(c)$. ΣMin is a minimal DNF constraint equivalent to c.

3.4 THE CLP(𝔹) LANGUAGE

CLP(𝔻) denotes a logic programming language incorporating the constraint system 𝔻. A CLP(𝔻) program consists of a set of definite clauses over atomic formulæ. Uninterpreted relation symbols are used to construct atomic formulæ over terms from 𝔻, which we call *atoms*. A definite clause consists of a *head* and a *body*. The head of each clause must be an atom and the body consists of a set of zero or more atoms or constraints. Standard logic programming can be thought of as CLP(ℍ) where ℍ is the constraint system consisting of equality constraints on Herbrand terms. Herbrand terms are terms constructed from uninterpreted constant and function symbols. Most CLP languages combine ℍ with one or more other constraint systems. To simplify the following discussion, we restrict our attention to the strict meaning of CLP(𝔹) where

only Boolean terms are allowed in atoms. To generalize the discussion for combined constraint systems, such as $\mathbb{H} \cup \mathbb{B}$, the sorts of terms and constraints must be considered.

As a running example, we use the following CLP($\mathbb{B}$) program consisting of a single clause:

$$\mathsf{p(A, B) \leftarrow A \Rightarrow X, X \Rightarrow B.} \tag{10}$$

The atom $\mathsf{p(A, B)}$ is the head of (10) and the two constraints $\mathsf{A \Rightarrow X}$ and $\mathsf{X \Rightarrow B}$ form the body. In general, the arguments to the relation symbol p could be arbitrary Boolean terms and the body could contain atoms. A clause can be read as "the head is true if all the elements of the body are true." A *query* is a clause with no head. For instance,

$$\mathsf{\leftarrow p(R{\cdot}S, T').} \tag{11}$$

is a query. Informally, a query asks if its body is true. For any query Q, we use $vars(Q)$ to denote the set of variable symbols in Q.

The operational semantics of a CLP($\mathbb{B}$) program is based on a generalization of SLD-refutation (Lloyd, 1984). A derivation state is a pair (R, C) where R is a set of atomic formulæ (atoms or constraints) and C is a set of constraints. A generalized SLD-refutation of a query Q = '$\leftarrow f_1, \ldots f_k$' with respect to a program P is a sequence

$$(R_1, C_1) = (\{f_1, \ldots f_k\}, \emptyset) \rightarrow (R_2, C_2) \rightarrow \ldots \rightarrow (R_n, C_n) = (\emptyset, A) \tag{12}$$

of derivation states which satisfies the following two conditions. First, ΠC_i $(1 \leq i \leq n)$ is satisfiable. Second, (R_{i+1}, C_{i+1}) is derived from (R_i, C_i) $(1 \leq i < n)$ by selecting a formula f from R_i and proceeding in one of two ways. If f is a constraint, then

$$(R_{i+1}, C_{i+1}) = (R_i \backslash \{f\}, C_i \cup \{f\}). \tag{13}$$

$$\begin{array}{lcl}
(\{\mathsf{p(R{\cdot}S, T')}\}, & \emptyset &)\rightarrow \\
(\{\mathsf{A \Rightarrow X,\ X \Rightarrow B}\}, & \{\mathsf{A = R{\cdot}S, B = T'}\} &)\rightarrow \\
(\{\mathsf{X \Rightarrow B}\}, & \{\mathsf{A = R{\cdot}S, B = T', A \Rightarrow X}\} &)\rightarrow \\
(\emptyset, & \{\mathsf{A = R{\cdot}S, B = T', A \Rightarrow X, X \Rightarrow B}\} &)
\end{array}$$

Figure 1 A refutation of (11) with respect to (10)

If $f = \mathsf{p}(t_1,\ldots,t_m)$ is an atom then there is a new variant[7]

$$\mathsf{p}(s_1,\ldots,s_m) \leftarrow b_1,\ldots,b_r.$$

of a clause in P, and

$$(R_{i+1}, C_{i+1}) = ((R_i \setminus \{f\}) \cup \{b_1,\ldots,b_r\}, C_i \cup \{t_1{=}s_1,\ldots,t_m{=}s_m\}). \tag{14}$$

ΠA is called the *answer* constraint *induced* by the refutation of Q with respect to the program P. For instance, figure 1 shows a refutation of query (11) with respect to program (10). The answer constraint induced by the refutation is $\Pi\{\mathsf{A} = \mathsf{R}\cdot\mathsf{S}, \mathsf{B} = \mathsf{T}, \mathsf{A} \Rightarrow \mathsf{X}, \mathsf{X} \Rightarrow \mathsf{B}\}$.

4 The Implementation of Boole

To implement a CLP(𝔹) interpreter, there must be a constraint satisfiability algorithm. In practice, this algorithm should be *incremental* and be adapted to *simplify* the answer constraint induced by a refutation. Section 4.1 shows why the satisfiability algorithm should be incremental and describes how the CM can be used to implement it. Section 4.2 shows how an efficient algorithm for finding simplified answer constraints can be implemented by augmenting the CM with a CMS.

4.1 Incremental Satisfiability

An incremental satisfiability algorithm is desirable for efficiency. Let C_i and $C_{i+1} = C_i \cup \{c\}$ be constraint sets associated with two consecutive derivation states where the constraint c was selected to make the transition from state i to state i+1. An incremental satisfiability algorithm, which has shown that ΠC_i is satisfiable, should be able to test if ΠC_{i+1} is satisfiable without redoing any work involved in testing the satisfiability of ΠC_i. Boole implements an incremental satisfiability test by operating on the matrix ΣN

[7]A new variant of a clause is a copy of that clause with each variable symbol replaced by a variable symbol that has not previously been used in the derivation. Every occurrence of a variable symbol must be replaced by the same new symbol in the new variant.

equivalent to $(\Pi C_i)'$ and the matrix n equivalent to c'. Recall that a constraint is satisfiable if and only if its negation is not valid and a matrix is not valid if and only if it contains a UC path. Consequently, Boole represents the fact that ΠC_i is satisfiable by a UC path U through the matrix ΣN. All CM validity algorithms systematically search for a UC path U through a matrix. Boole saves U, which proves ΠC_i is satisfiable. Then, to test the satisfiability of ΠC_{i+1}, Boole searches for a UC path $U \cup V$ where V is a UC path through n. If necessary, Boole changes U by backtracking the search into ΣN. $U \cup V$ is a UC path through $\Sigma(N \cup \{n\})$, which is equivalent to $(\Pi C_{i+1})'$. Thus, $U \cup V$ proves the satisfiability of ΠC_{i+1}.

For example, consider the transition from the third to the fourth derivation state in figure 1. The constraint set associated with the third derivation state is $C_3 = \{A = R \cdot S, B = T', A \Rightarrow X\}$. A matrix equivalent to $(\Pi C_3)'$ is[8]:

$$\Sigma N = \left[\begin{array}{c|c|c|c|c} \begin{array}{c} A \\ \hline R' \ S' \end{array} & \begin{array}{c} A' \\ \hline R \\ S \end{array} & \begin{array}{c} B \\ T \end{array} & \begin{array}{c} B' \\ T' \end{array} & \begin{array}{c} A \\ X' \end{array} \end{array}\right].$$

Boole finds the UC path $U = \{A, R, B, T'\}$[9], which shows ΠC_3 is satisfiable. To construct the fourth derivation state, it is necessary to show ΠC_4 is satisfiable where $C_4 = C_3 \cup \{X \Rightarrow B\}$. The matrix equivalent to $(X \Rightarrow B)'$ is:

$$n = \left[\begin{array}{c} X \\ \hline B' \end{array}\right].$$

Extending U with the UC path $V = \{X\}$ through n shows ΠC_4 is satisfiable, since $U \cup V$ is a UC path through $\Sigma(N \cup n)$. $U \cup V$ is a proof of the satisfiability of ΠC_4 which can be constructed without redoing any of the work in showing the satisfiability of ΠC_3. To illustrate how U may need to be changed by backtracking, consider what would happen if n had instead been $A' \cdot B'$. Since both extensions of U through n are complementary, the

[8]This example assumes constraints of the form $t_1 = t_2$ are converted to equivalent matrices in the same way constraints of the form $t_1 \Leftrightarrow t_2$ are converted using [2]. Boole actually converts constraints of the form $V = t$ where $V \in$ Var and $t \in$ Term by substituting occurrences of V for t in all future derivation states.

[9]The literal A is actually encountered twice while generating U. It only appears once in U since U is a set.

search for UC paths must backtrack and try a different path through ΣN. If U is changed to {A, R, T, B'}, it can be extended with {B'} to form a UC path through $\Sigma(N \cup n)$.

4.2 Simplifying Answer Constraints

The answer constraint should be in a form suitable for output as the answer to the query. It should make the relationship between the variables in that query explicit and contain a small number of symbols. Moreover, it need only contain variable symbols in the query. For instance, the answer constraint $\Pi\{A = R \cdot S, B = T', A \Rightarrow X, X \Rightarrow B\}$, from the refutation in figure 1, should be output in some *simplified answer form* such as R'+S'+T' or $R \cdot S \Rightarrow T'$. Notice that the variable symbols A, B, and X, which do not appear in the query, can be dropped from the answer constraint without losing any information. All that matters is that there exist values for A, B, and X such that any set of values for R, S, and T which make the query true make the simplified answer constraint true.

More formally, let Q be a query and ΠA be the answer constraint induced by the refutation of Q. This section shows how to compute a simplified answer constraint a such that:

A1. a is in a minimal DNF,

A2. $vars(a) \subseteq vars(Q)$, and

A3. a is equivalent to $\exists(vars(\Pi A) \backslash vars(Q))\ \Pi A$.

The constraint a is often a good representation of ΠA. Since it is in minimal DNF form, it often contains a very small number of function symbols. Informally, the constraint $\exists(vars(\Pi A) \backslash vars(Q))\ \Pi A$ is true whenever ΠA is true disregarding the variables not used in Q. Because it only uses variable symbols used in Q but is equivalent to $\exists(vars(\Pi A) \backslash vars(Q))\ \Pi A$, it contains all the information which can be expressed in terms of the variable symbols in Q.

Boole simplifies answer constraints with a CMS based problem solving architecture. Reiter & de Kleer (1987) give a foundation for CMS architectures. They propose a problem solving architecture which consists of a task specific reasoner which communicates with a general CMS. The reasoner uses the CMS to record facts it has discovered and the CMS stores these facts in a compiled form. The reasoner may also

query the CMS to determine the status of facts it has discovered. Reiter and de Kleer show that if the facts and queries are always clauses then compiling the facts to prime implicants allows the CMS to give useful answers to the reasoner's queries. These answers allow the reasoner to avoid searching for facts which are implied by facts it has already discovered and transmitted to the CMS[10]. The reasoner in Boole's CMS architecture contains a CM theorem prover and the CMS is implemented with the incremental prime implicant generation algorithm of Kean & Tsiknis (1990).

We start by describing a basic procedure which simplifies answer constraints so they satisfy A1. Then, we refine the basic procedure to make it more efficient and simplify answer constraints so they satisfy A1-A3. Let Q be a query, ΠA be the answer constraint induced by the refutation of Q, and m be a matrix equivalent to ΠA. A CM theorem prover can be modified to incrementally generate the set:

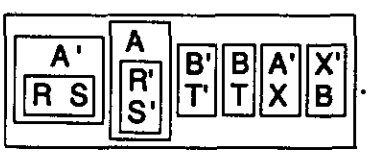
$$UCpaths(dual(m)) = \{L_1,\ldots,L_p\}.$$

Corollary 1 in the appendix establishes that:

$$I = \{\Pi L_1,\ldots,\Pi L_p\}$$

is a complete set of implicants for ΠA. The basic procedure is to have the reasoner transmit I to the CMS. Then, the CMS stores the set $PI(\Pi A)$ and the Quine-McCluskey algorithm (McCluskey, 1986) is used to select a non-redundant subset of the prime implicants in the CMS which constitutes a minimal DNF constraint equivalent to ΠA.

For example, let Q be the query (11) with respect to program (10). The answer constraint induced by the refutation of Q is ΠA where $A = \{A = R{\cdot}S, B = T', A \Rightarrow X, X \Rightarrow B\}$. The dual of the matrix m equivalent to ΠA is:

$dual(m) =$ 

.

Thus, we have:

[10]It should be pointed out that we use the dual of Reiter and de Kleer's specification. They use disjunctive clauses and prime implicates whereas we use conjunctive clauses and prime implicants.

$$UCpaths(dual(m)) = \{ \{A',R',B',T,X'\}, \{A',R',T',B,X'\}, \{A',R',T',B\}, \{A',R',T',B,X\}, \{A',S',B',T,X'\}, \{A',S',T',B,X'\}, \{A',S',T',B\}, \{A',S',T',B,X\}, \{R,S,A,T',B,X\} \}.$$

Let the elements of this set be $\{L_i \mid 1 \le i \le 9\}$. The basic procedure transmits the set:

$$I = \{\Pi L_1,\ldots,\Pi L_9\}$$

to the CMS. The CMS then contains

$$PI(\Pi A) = \{A{\cdot}B{\cdot}R{\cdot}S{\cdot}T'{\cdot}X, A'{\cdot}B{\cdot}S'{\cdot}T', A'{\cdot}B{\cdot}R'{\cdot}T', A'{\cdot}B'{\cdot}S'{\cdot}T{\cdot}X', A'{\cdot}B'{\cdot}R'{\cdot}T{\cdot}X'\}.$$

In this case, $PI(\Pi A)$ is non-redundant, so the minimal DNF constraint equivalent to ΠA is:

$$A{\cdot}B{\cdot}R{\cdot}S{\cdot}T'{\cdot}X + A'{\cdot}B{\cdot}S'{\cdot}T' + A'{\cdot}B{\cdot}R'{\cdot}T' + A'{\cdot}B'{\cdot}S'{\cdot}T{\cdot}X' + A'{\cdot}B'{\cdot}R'{\cdot}T{\cdot}X'.$$

This basic procedure has two problems. First, since I can be very expensive to compute and it may be highly redundant, the basic procedure can be very inefficient. Second, it only simplifies ΠA to a constraint satisfying A1. The simplified answer constraint above, which does not satisfy A2-A3, makes it difficult to see any relationship between the variable symbols in the query. The remainder of the section, we modify the basic procedure to address the two problems.

The reasoner can be more efficient by using the CMS to avoid generating redundant implicants, reducing its search space. Let $L_i = \{l_{i1},\ldots,l_{iq_i}\}$ $(1 \le i \le p)$. Since L_i is generated one literal at a time by searching in the answer constraint, the reasoner can determine if a partially generated implicant $\Pi\{l_{i1},\ldots,l_{ir_i}\}$ $(r_i < q_i)$ is redundant with respect to the previously generated implicants by querying the CMS. If the partial implicant $\Pi\{l_{i1},\ldots,l_{ir_i}\}$ is subsumed by a prime implicant in the CMS, by the definition of prime implicants, it is subsumed by ΠL_j for some $j < i$. Thus, the full implicant ΠL_i is redundant with the implicants already transmitted to the CMS. In this case, the reasoner can avoid the effort required to finish generating ΠL_i. To see this, reconsider the example above after the implicants L_1, L_2, and L_3 have been transmitted to the CMS. At this point, the CMS contains the set:

$$PI(\Sigma\{L_1, L_2, L_3\}) = \{\Pi\{A', R', B', T, X'\}, \Pi\{A', R', T', B\}\}.$$

The reasoner does not need to fully generate L_4 because querying the CMS with the implicant formed from its first four literals, $\Pi\{A', R', T', B\}$, reveals that L_4 is subsumed by the prime implicant $\Pi\{A', R', T', B\}$ already stored in the CMS.

If ΠA is valid, then eventually the CMS will contain the single implicant $\Pi\emptyset = 1$. When this is the case, the reasoner can stop generating implicants and output the answer because 1 subsumes all implicants.

To compute an answer constraint satisfying A2-A3, the reasoner uses the same procedure but deletes from implicants the literals which involve variables not in $vars(Q)$. That is, the reasoner uses the set:

$$J = \{\Pi K_1, \ldots, \Pi K_p\}$$

of implicants where $K_i = L_i \cap \{V, V' \mid V \in vars(Q)\}$ $(1 \leq i \leq p)$. Theorem 2 in the appendix establishes that ΣJ is equivalent to $\exists(vars(\Sigma J) \backslash vars(Q))\ \Sigma J$, which is equivalent to $\exists(vars(\Pi A) \backslash vars(Q))\ \Pi A$ since I is a complete set of implicants for ΠA. Thus, the constraint a, which is the sum of a non-redundant subset of $PI(\Sigma J)$ eventually stored in the CMS, satisfies A1-A3.

Using the above example, the reasoner only needs to transmit the set:

$$J = \{\Pi\{R', T\}, \Pi\{R', T'\}, \Pi\{S', T\}, \Pi\{S', T'\}, \Pi\{R, S, T'\}\}.$$

Then, the CMS contains $PI(\Sigma J) = \{R', S', T'\}$ and the simplified answer constraint satisfying A1-A3 is $R'+S'+T'$.

5 Conclusions and Future Work

Boole implements CLP(B) using relatively simple modifications to a CM theorem prover and a CMS. This paper gave the necessary background to formally describe and study Boole's implementation. A first prototype of Boole has been implemented in Quintus Prolog. This prototype is implemented without any special data structures and the CM theorem prover is implemented by naïvely axiomatizing path rules P1-P3 given in section 3.2. Comparisons with other systems show Boole promises to be efficient and practical.

For constraints which have a very large CNF or which are equivalent to matrices containing very few UC paths, the prototype actually outperforms Prolog III.

The prototype shows two weaknesses of the Boole approach to CLP(**B**). The first weakness is as follows. When the answer constraint, ΠA, for a query Q has many UC paths, simplification can be very expensive. This problem may be solved by transmitting implicants to the CMS involving variables outside $vars(\Pi A) \backslash vars(Q)$ or searching in $\exists(vars(\Pi A) \backslash vars(Q))\Pi A$ directly. Both of these possibilities need to be explored further.

The second weakness is evident when the minimal DNFs of the answer constraint are complicated yet there is some other simple form for the answer constraint. For instance, the minimal DNF of the constraint $(X \Leftrightarrow B) \cdot (Y \Leftrightarrow A)$ is $A \cdot B \cdot X \cdot Y + A \cdot B' \cdot X' \cdot Y + A' \cdot B \cdot X \cdot Y' + A' \cdot B' \cdot X' \cdot Y'$. There are several potential approaches to solving this problem. It may be possible to generalize the notion of implicants to matrices, but this idea needs further development. In general, it is a matter of personal preference which expression makes the meaning of a solution constraint most clear. A symbolic manipulation system like Mathematica (Wolfram, 1991) would help a user to discover this representation.

Although the first prototype has some weaknesses, it performs quite well considering the simplicity of its implementation. Consequently, in the future a similar but more efficient implementation of Boole will be built which addresses the weaknesses of the current implementation.

Acknowledgments

This work was supported by the Centre for Systems Science at Simon Fraser University, by the Alberta Research Council, and by PRECARN Associates. The author would like to thank Veronica Dahl for supervising this work, and Jeorg Ueberla and Sue Sidebottom for pointing out errors in earlier drafts of this paper.

Appendix: Theorems Justifying the Simplification Procedure

In this appendix, we prove some results which justify some of the steps in Boole's simplification procedure.

First, we prove that the set of UC paths through the dual of a matrix are a complete set of implicants for that matrix. We do this using the following generalization of a lemma in (Murray & Rosenthal, 1987):

THEOREM 1 *A grounding substitution σ satisfies a matrix m if and only if σ satisfies ΠP for some $P \in UCpaths(dual(m))$.*

PROOF: Murray and Rosenthal (1987) prove this result except they consider $paths(dual(m))$ instead of $UCpaths(dual(m))$[11]. To extend the result for $UCpaths(dual(m))$, let σ be a grounding substitution and note that σ satisfies no ΠP where $P \in paths(dual(m)) \backslash UCpaths(dual(m))$. This is because P contains a complementary pair of literals so ΠP is unsatisfiable. Thus, if σ satisfies m then σ satisfies ΠP for some path $P \in UCpaths(dual(m))$. Conversely, if σ satisfies ΠP for some $P \in UCpaths(dual(m))$ then, since $UCpaths(dual(m)) \subseteq paths(dual(m))$, Murray and Rosenthal's result applies and σ satisfies m. □

COROLLARY 1 *For any matrix m with $UCpaths(dual(m)) = \{L_1,\ldots,L_p\}$, $\{\Pi L_1,\ldots,\Pi L_p\}$ is a complete set of implicants for m.*

PROOF: Let σ be a grounding substitution. Thus, by theorem 1, if σ satisfies m then σ satisfies L_k for some k ($1 \leq k \leq p$). Thus, σ satisfies $\Sigma\{\Pi L_1,\ldots,\Pi L_p\}$, meaning m is equivalent to $\Sigma\{\Pi L_1,\ldots,\Pi L_p\}$. Consequently, $\{\Pi L_1,\ldots,\Pi L_p\}$ is a complete set of implicants for m. □

Next, we prove that the set of UC paths through the dual of a matrix is, in a sense, equivalent to that matrix.

---

[11]Note that Murray and Rosenthal call matrices *semantic graphs*, grounding substitutions *interpretations*, and matrices of the form ΠP where $P \in paths(dual(m))$ *c-paths*.

COROLLARY 2 *For any matrices m_1 and m_2, paths(dual(m_1)) = paths(dual(m_2)) if and only if m_1 is equivalent to m_2.*

PROOF: Since $paths(dual(m_1)) = paths(dual(m_2))$, $UCpaths(dual(m_1)) = UCpaths(dual(m_2))$. Let $UCpaths(dual(m_1)) = UCpaths(dual(m_2)) = \{L_1,\ldots,L_p\}$. By theorem 1, $\Sigma\{\Pi L_1,\ldots,\Pi L_p\}$ is equivalent to m_1 and to m_2. Thus, m_1 and m_2 are equivalent. □

We use corollary 2 in the proof of the following theorem, which justifies dropping variables from implicants which are not in the query.

THEOREM 2 *Let $I = \{\Pi L_1,\ldots,\Pi L_p\}$ be a set of clauses, $V \subseteq vars(\Sigma I)$, $W = vars(\Sigma I)\backslash V$, and $J = \{\Pi K_1,\ldots,\Pi K_p\}$ be such that $K_i = L_i \cap \{X, X' \mid X \in V\}$ for all i $(1 \le i \le p)$. Then ΣJ is equivalent to $\exists W\ \Sigma I$.*

PROOF: First, let us consider the special case where $W = \emptyset$. Then $V = vars(\Sigma I)$, so $K_i = L_i$ $(1 \le i \le p)$ and thus $I = J$. Since $\exists\emptyset\ \Sigma I = \Sigma I$ by definition, ΣJ is equivalent to $\exists W\ \Sigma I$.

Now consider the case where $W \neq \emptyset$. In this case, we prove the theorem by showing that $paths(dual(\Sigma J)) = paths(dual(\exists W\ \Sigma I))$. First note that:

$$paths(dual(\Sigma J)) = \{K_1,\ldots,K_p\}.$$

Let $L_i = \{l_{i1},\ldots,l_{iq_i}\}$ $(1 \le i \le p)$. By applying the definition of '$\exists$', we see that $dual(\exists W\ \Sigma I)$ is the conjunction of the result of r $(= 2^{|W|})$ substitutions on $dual(\Sigma I)$, one for each possible way of substituting Boolean constants for the variables in W. Replacing each occurrence of 0' by 1 and each occurrence of 1' by 0, we see that $dual(\exists W\ \Sigma I)$ is equivalent to the matrix $\Pi\{z_1,\ldots,z_r\}$ where:

$$z_k = \begin{bmatrix} \boxed{l^k_{11},\ldots,l^k_{1q_1}} \\ \vdots \\ \boxed{l^k_{p1},\ldots,l^k_{pq_p}} \end{bmatrix},$$

and each $l^k_{ij_i}$ $(1 \le i \le p,\ 1 \le j_i \le q_i,\ 1 \le k \le r)$ is either the literal l_{ij_i} if l_{ij_i} involves a variable from V or a Boolean constant otherwise. Potential paths through $\Pi\{z_1,\ldots,z_r\}$ are subsets of the rows $L^k_i = \{l^k_{i1},\ldots,l^k_{iq_i}\}$ $(1 \le i \le p,\ 1 \le k \le r)$. Since there is one z_k for each possible substitution of the variables in W, for each i there is exactly one k such that $1 \notin L^k_i$. Given i, we denote such a k by $f(i)$. Informally, the substitution given by $f(i)$ sets false all the literals in L_i involving variables from W. Now we can see that:

$$paths(\Pi\{z_1,\ldots,z_r\}) = \{L^{f(i)}_i \setminus \{0\} \mid 1 \le i \le p\}.$$

This is because since $paths(1) = \emptyset$, paths through rows containing 1's are blocked and since $paths(0) = \{\emptyset\}$, the path through each $L^{f(i)}_i$ is $L^{f(i)}_i \setminus \{0\}$. Now since $L^{f(i)}_i$ is the result of substituting 0's for literals involving variables from W, $K_i = L^{f(i)}_i \setminus \{0\}$ for all i $(1 \le i \le p)$. Thus, $paths(dual(\Sigma J)) = paths(\Pi\{z_1,\ldots,z_r\})$. Since $\Pi\{z_1,\ldots,z_r\}$ is equivalent to $dual(\exists W\ \Sigma I)$, by corollary 2, we have $paths(\Pi\{z_1,\ldots,z_r\}) = paths(dual(\exists W\ \Sigma I))$. Again by corollary 2, since $paths(\Pi\{z_1,\ldots,z_r\}) = paths(dual(\exists W\ \Sigma I))$, we have ΣJ is equivalent to $\exists W\ \Sigma I$. □

References

Bibel, W. (1983) "Matings in Matrices," *Comm. ACM*, 26 (11), 844-852.

Bibel, W. (1987) *Automated Theorem Proving*, Artificial Intelligence, W. Bibel (ed.), Vieweg, Braunschweig..

Boi, J. M., Benhamou, F. (1988) "Boolean Constraints in Prolog III." Ph.D., Groupe d'Intelligence Artificielle, Universite d'Aix-Marseille, Luminy.

Buttner, W., Simonis, H. (1987) "Embedding Boolean Expressions into Logic Programming," *J. Symbolic Computation*, 4, 191-205.

Colmerauer, A. (1984) "Equations and Inequations on Finite and Infinite Trees." In Proc. *Fifth Generation Computer Systems*, 85-99.

Colmerauer, A. (1990) "An Introduction to Prolog III," *Comm. ACM*, 33 (7), 69-90.

Dincbas, M., Van Hentenryck, P., Simonis, H., Aggoun, A. (1988) "The Constraint Logic Programming Language CHIP." In Proc. *The International Conference on Fifth Generation Computer Systems*, Omhsha Publishers, Tokyo, 693-702.

Jaffar, J., Lassez, J.-L. (1987) "Constraint Logic Programming." In Proc. *Fourteenth ACM ACM Symposium on the Principles of Programming Languages*, Munich, 111-119.

Jaffar, J., Michaylov, S. (1987) "Methodology and Implementation of a CLP System." In Proc. *Fourth International Conference on Logic Programming*, Melbourne, Australia, 111-119.

Kean, A., Tsiknis, G. (1990) "An incremental Method for Generating Prime Implicants," *Journal of Symbolic Computation*, **9** (2), 185-206.

Lloyd, J. W. (1984) *Foundations of Logic Programming*, Symbolic Computation, D. W. Loveland (ed.), Springer-Verlag, New York..

McCluskey, E. J. (1986) *Logic Design Principles*, Prentice-Hall, Englewood Cliffs, NJ..

Murray, N. V., Rosenthal, E. (1987) "Inference with Path Resolution and Semantic Graphs," *J. Association for Computing Machinery*, **34** (2), 225-254.

Reiter, R., de Kleer, J. (1987) "Foundations of Assumption-Based Truth Maintenance Systems: Preliminary Report." In Proc. *National Conference on Artifical Intelligence (AAAI-87)*, Seattle, 183-188.

Sakai, K., Aiba, A. (1989) "CAL: A Theoretical Background of Constraint Logic Programming and its Applications," *J. Symbolic Computation*, **8** (6), 589-603.

Saraswat, V. A. (1990) *Concurrent Constraint Programming Languages*, MIT Press, Cambridge MA.

Van Hentenryck, P. (1989) *Constraint Satisfaction in Logic Programming*, The MIT Press, Cambridge..

Walinsky, C. (1989) "Constraint Logic Programming with Regular Sets." In Proc. *Sixth International Conference on Logic Programming*, Lisbon, Portugal, 181-196.

Wolfram, S. (1991) *Mathematica: A System for Doing Mathematics by Computer*, Addison-Wesley, Don Mills, ON.